Aula 4

Escola Politécnica

Arquitetura de Sistemas

Profª Drª Mariane Gavioli Bergamini Fernandes

Conversa Inicial

- Os objetivos da aula serão: compreender os fundamentos teóricos sobre design patterns, contextualizar os tipos de padrões utilizados e compreender e avaliar os padrões de reusos mais utilizados.

- Durante nossa aula será explorado 5 temas:
- Aspectos fundamentais sobre padrões de projeto reutilizáveis (Design Patterns).
- Padrões criacionais
- Padrões estruturais
- Padrões comportamentais
- Aprofundamento nos padrões mais usados

ASPECTOS FUNDAMENTAIS SOBRE PADRÕES DE PROJETO REUTILIZÁVEIS

Padrão Arquitetônico

- Padrão arquitetônico fornece um conjunto de subsistemas, assim como as responsabilidade, regras e diretrizes para compreender a interação entre eles.

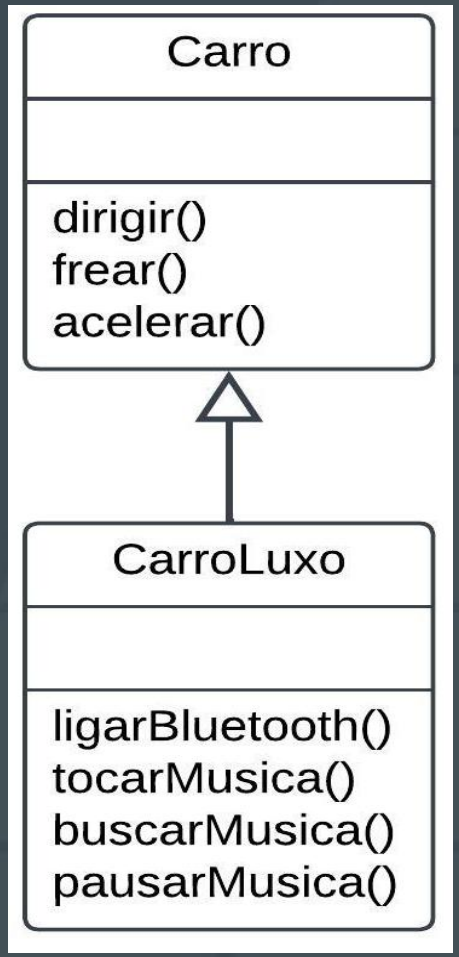
- Um conjunto de componentes
- Responsabilidade dos componentes envolvidos (papéis)
- Relacionamentos entre componentes

Design Patterns

- Padrões de design podem ser considerados como soluções conceituais para problemas recorrentes de design.
- Cada padrão trata um problema no design ou na implementação do software.

Reuso

- Ser capaz de lidar com novos requisitos, evitar o redesenho ou minimizá-lo.
- Uma vantagem é utilizar técnicas orientadas a objetos, podem ser mais flexíveis.



Herança → objetos de solução adicionados → bibliotecas de classes → framework.

PADRÕES CRIACIONAIS

Padrão Criacional

- Garante a modulariedade do software
- Menor dependência de classes concretas

- Instanciação de classe: define a melhor maneira na qual um objeto pode ser instanciado.
- Criação de classe: Utiliza a herança no processo de instanciação.
- Criação de objeto: Utiliza a delegação para realizar o trabalho.

Tipos de padrões criacional

TIPO	CRIACIONAL
Classe	Factory method
Objetivo	Abstract factory Builder Prototype Singleton

Padrão Factory

- Fornece uma interface para desenvolver objetos
- Permite que subclasses modifiquem o tipo de objetos que são desenvolvidos

Exemplo Factory Method

- Centralizar a criação de objetos de diferentes classes derivadas de uma classe base. Em vez de criar objetos diretamente utilizando '*new*', uma função '*createProduct*' decide qual classe instanciar com base no '*ProductID*'.

PADRÕES ESTRUTURAIS

Padrão de Design Estrutural

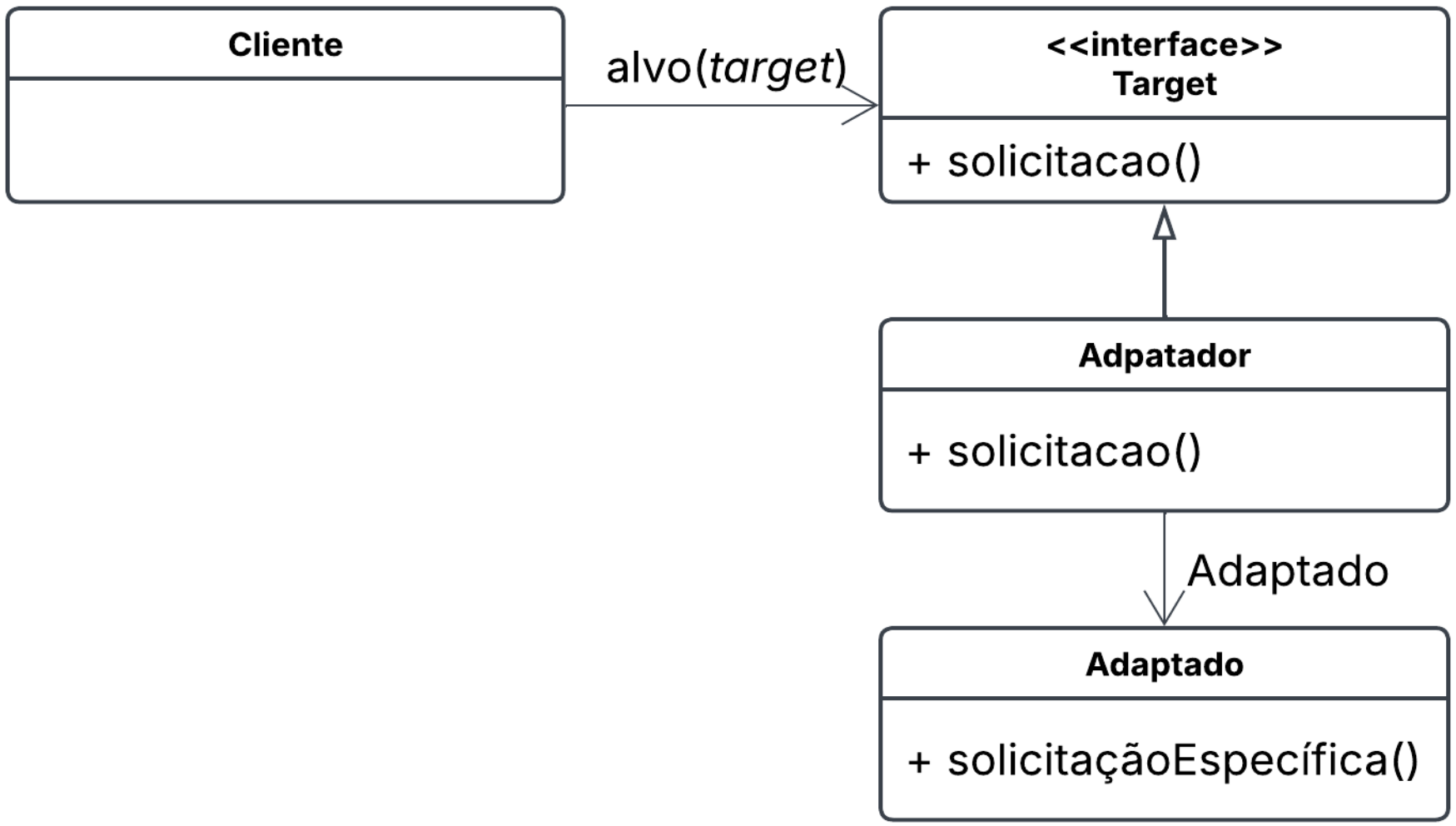
- Lida com a composição de classes ou de objetos.
- Classes e objetos são compostos
- Herança para compor interfaces

Tipos de padrões estruturais

- Adaptador (*Adapter*)
- Ponte (*Bridge*)
- Composto (*Composite*)
- Decorador (*Decorator*)
- Fachada (*Facade*)

Padrão Adapter

- A classe adaptador é usada para traduzir solicitações dela para outra classe (ou seja, o adaptado).
  - ✓Adaptar a classe
  - ✓Adaptar o objeto


Cliente
alvo(*target*)
<<interface>>
Target
+ solicitacao()
Adpatador
+ solicitacao()
Adaptado
Adaptado
+ solicitaçãoEspecífica()

PADRÕES COMPORTAMENTAIS

Padrões comportamentais

- Descrevem não apenas padrões de objetos ou classes, mas também os padrões de comunicação entre eles.

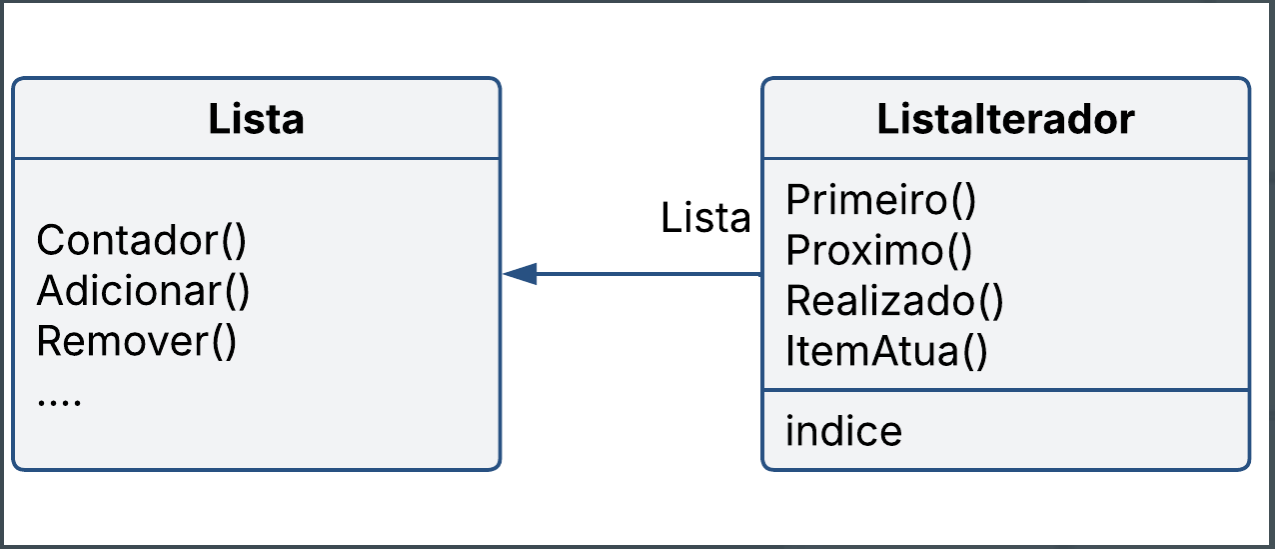
Padrões comportamentais

- Estratégia (*strategy*)
- *Template*
- Observador (*observer*)
- Iterador (*iterator*)
- Estado (*state*)

Padrão comportamental iterador

- Promove o desacoplamento, permitindo que algoritmos de iteração sejam independentes da estrutura de dados específica.

- A implementação de "ListIterator" abstrai os detalhes de como a lista é armazenada.



- A principal ideia do padrão comportamental é desacoplar remetentes e destinatários fornecendo a vários objetos uma chance de lidar com uma solicitação.
- Aumenta a flexibilidade na execução da comunicação.

APROFUNDAMENTO NOS PADRÕES MAIS USADOS

MVC com padrão Observador

Cenário: Para uma aplicação web MVC, vamos imaginar um sistema de gerenciamento de tarefas.

- Quais são as tarefas dos componentes modelo, visão e controlador?

Cliente-servidor com padrão Iterador

- Cenário: Imagine um sistema de gerenciamento de projetos de software. O servidor armazena informações sobre vários projetos, cada um contendo tarefas, equipes e documentação.
- O motivo de se utilizar o padrão iterador neste cenário?

Microsserviços com padrão fábrica (*factory*)

- Cenário: Um microsserviço responsável pelo processamento de pagamentos, onde precisa lidar com diferentes gateways de pagamento, como PayPal, Stripe e PagSeguro. Cada gateway tem suas próprias APIs e requisitos de integração.

Padrões de design

- Arquitetura MVC utilizando o padrão observador
- Arquitetura cliente servidor utilizando o padrão iterador
- Arquitetura microsserviços utilizando o padrão fábrica (*factory*).

Fechar